Task A5 – Design for Correctness and Safety (4 Marks, K4)

Part A asks for design, pseudocode and reasoning, so the answer is in pseudocode and tables. The Java in the *Alternative* sections is only for illustration. Items marked `<fill>` need your own observations.

A5.1 – Transaction design: transfer ₹500 from hostel wallet to canteen wallet

```
PROCEDURE transfer(con, hostelWalletId, canteenWalletId, amount = 500)
    oldAutoCommit ← con.getAutoCommit()
    con.setAutoCommit(false)                                    //
start one transaction
    TRY
        rows ← EXECUTE  "UPDATE wallet SET balance = balance - ? WHERE
id = ? AND balance >= ?"   (amount, hostelWalletId, amount)
        IF rows ≠ 1 THEN raise "insufficient balance or unknown
wallet"
        // a failure or crash here is safe: nothing is permanent yet
        rows ← EXECUTE  "UPDATE wallet SET balance = balance + ? WHERE
id = ?"                    (amount, canteenWalletId)
        IF rows ≠ 1 THEN raise "unknown canteen wallet"
        con.commit()                                            //
both updates become permanent together
    CATCH any error
        con.rollback()                                          // the
debit is undone
        RE-RAISE the error
    FINALLY
        con.setAutoCommit(oldAutoCommit)
END PROCEDURE
```

Why money is never lost or created: either both updates are committed (the total in the two wallets is unchanged), or neither is (rollback). The `balance >= ?` condition in the debit prevents negative balances. Java version (compiled): `Transfer.java`.

```java
import java.sql.*;

public class Transfer {
    /** Moves 'amount' from the hostel wallet to the canteen wallet
atomically. */
    public static void transfer(Connection con, String hostelId,
String canteenId, double amount) throws SQLException {
        boolean old = con.getAutoCommit();
        con.setAutoCommit(false);
// begin transaction
        try (PreparedStatement debit  = con.prepareStatement("UPDATE
wallet SET balance = balance - ? WHERE id = ? AND balance >= ?");
             PreparedStatement credit = con.prepareStatement("UPDATE
wallet SET balance = balance + ? WHERE id = ?")) {
```

```
            debit.setDouble(1, amount); debit.setString(2, hostelId);
debit.setDouble(3, amount);
            if (debit.executeUpdate() != 1) throw new
SQLException("Insufficient balance or unknown wallet");
            // <-- a crash here is safe: nothing is permanent until
commit()
            credit.setDouble(1, amount); credit.setString(2,
canteenId);
            if (credit.executeUpdate() != 1) throw new
SQLException("Unknown canteen wallet");
            con.commit();
// both updates become permanent together
        } catch (SQLException | RuntimeException e) {
            con.rollback();
// undo the debit
            throw e;
        } finally {
            con.setAutoCommit(old);
        }
    }
}
```

A5.2 – Injection analysis

```
String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" +
pwd + "'";
```

User input is pasted into the SQL text, so input can change the *structure* of the query. Strings produced by `Login.java` (sandbox):

```
id=arun   pwd=x' OR '1'='1
 → SELECT * FROM users WHERE id='arun' AND pwd='x' OR '1'='1'
```

`AND` binds tighter than `OR`, so the condition is `(id='arun' AND pwd='x') OR '1'='1'`. `'1'='1'` is always true, so the query returns **every row** and the login succeeds without a valid password. Another input: id = `admin'#` makes the rest of the line a comment in MySQL:

```
 → SELECT * FROM users WHERE id='admin'#' AND pwd='anything'
(password check commented out)
```

Safe redesign with `PreparedStatement`:

```
try (PreparedStatement ps = con.prepareStatement("SELECT 1 FROM users
WHERE id = ? AND pwd = ?")) {
    ps.setString(1, id);
    ps.setString(2, pwd);
    try (ResultSet rs = ps.executeQuery()) { return rs.next(); }
}
```

The SQL text with the `?` placeholders is parsed first and fixed; the values are sent separately as *data*, so a quote in `x' OR '1'='1` is just part of the password string being compared, and the query finds no match. (In real systems also store salted password hashes, not plain passwords.)

```java
import java.sql.*;

public class Login {
    /** UNSAFE: user input becomes part of the SQL text. */
    static String unsafeSql(String id, String pwd) {
        return "SELECT * FROM users WHERE id='" + id + "' AND pwd='" +
pwd + "'";
    }
    /** SAFE: SQL structure is fixed; input is sent only as data. */
    static boolean safeLogin(Connection con, String id, String pwd)
throws SQLException {
        try (PreparedStatement ps = con.prepareStatement("SELECT 1
FROM users WHERE id = ? AND pwd = ?")) {
            ps.setString(1, id); ps.setString(2, pwd);
            try (ResultSet rs = ps.executeQuery()) { return rs.next();
}
        }
    }
    public static void main(String[] a) {
        System.out.println(unsafeSql("arun", "wrong"));
        System.out.println(unsafeSql("arun", "x' OR '1'='1"));
        System.out.println(unsafeSql("admin'#", "anything"));
    }
}
```

Alternative concept and complexity

Two separate auto-commit statements instead of one transaction:

```java
con.setAutoCommit(true);
stmt.executeUpdate("UPDATE wallet SET balance = balance - 500 WHERE id
= 'H1'");   // committed immediately
// ← program crashes / exception here
stmt.executeUpdate("UPDATE wallet SET balance = balance + 500 WHERE id
= 'C1'");   // never runs
```

- **Outcome on failure between the updates:** the debit is already permanent, the credit never happens, so **₹500 disappears** from the system (the student lost money, the canteen did not receive it). Nothing can undo it automatically.
- **Time cost:** a transaction groups both updates into one commit, so the database writes its log to disk once (one commit). Two auto-commit statements commit twice, so they usually cost at least as much and often *more*. A transaction's extra costs are holding locks longer and the rollback work on failure; for two updates that is small. So the transaction does not cost a significant amount of time: O(1) extra either way.
- **Space cost:** the transaction keeps undo information for the rows changed until commit: O(number of rows changed), here 2 rows; negligible.
- **Risk to data correctness:** with separate statements, any failure, crash, network drop or exception between them produces inconsistent data (money lost or created), and

concurrent readers can see the intermediate state. A transaction gives **atomicity** and isolation, so the correctness risk is removed.